

LAB 1 REPORT

TokTickIT Full-Stack IT Service Desk Starter

Metadata Key	Specification / Output
Course & Lab Name	CPE 334 — Lab 1: TokTickIT Full-Stack Hello World Starter
Assignment Type	Assignments / Homework
Author / Student GitHub	Vieng (@vienggg)
Peer Reviewer GitHub	Dechayut (@NinjoMUDA)
Project Repository URL	https://github.com/vienggg/toktickit
Partner Repository URL	https://github.com/NinjoMUDA/Dechayut_3414Lab1
GitHub Project Board URL	https://github.com/users/vienggg/projects/1
Lab Learning Outcomes	- Build a full-stack vertical slice: React/Vite/Bootstrap UI → Express API → Prisma/PostgreSQL. - Implement REST endpoints, automated testing (Vitest/Supertest), and Git Flow. - Utilize GitHub Projects for task tracking and peer-reviewed Pull Requests.

ANSWER PART 1: PROJECT FOUNDATION & ENVIRONMENT SETUP

Issue 1 established the full-stack foundation for TokTickIT. We installed npm package dependencies for both `server/` and `client/` modules, configured environment files (`.env`) from template examples (`.env.example`), verified TypeScript compilation without errors, and submitted Pull Request #1 from `feature/1-project-foundation` to `lab1-staging`.

 [SCREENSHOT PLACEHOLDER: GitHub Pull Request #1 & Git Flow Branch Setup]

Figure 1: Pull Request #1 merged into lab1-staging showing clean feature branch setup

ANSWER PART 2: API HEALTH CHECK IMPLEMENTATION

Issue 2 implemented the GET `/api/health` endpoint in `server/src/app.ts` returning HTTP 200 OK with JSON payload `{ status: "ok", service: "TokTickIT API" }`. The endpoint was validated using Supertest integration tests in `server/tests/lab-01/health.test.ts`, passing all 1 test suite check.

 [SCREENSHOT PLACEHOLDER: Supertest GET `/api/health` Terminal Execution Evidence]

Figure 2: Supertest terminal output verifying HTTP 200 response from `/api/health`

ANSWER PART 3: CATEGORY MODEL & IDEMPOTENT SEEDING

Issue 3 defined the Category model in `server/prisma/schema.prisma` (containing `id`, `name`, and `createdAt` fields) and implemented an idempotent seed script in `server/prisma/seed.ts` using Prisma's `upsert` API. The seed script inserts four standard IT request categories: "Account and Access", "Hardware", "Software", and "Network".

 [SCREENSHOT PLACEHOLDER: Prisma Category Model & Seed Terminal Execution]

Figure 3: Prisma schema definition and seed output populating 4 IT request categories

ANSWER PART 4: CATEGORY LIST API, REACT UI, & TESTING EVIDENCE

Issue 4 created the GET `/api/categories` endpoint returning seeded categories ordered by ID ascending. In the frontend, `client/src/api.ts` `checkSystem()` method fetches health and categories endpoints. The React UI in `client/src/App.tsx` presents a clean Bootstrap layout displaying Loading state, Success state ("Online" badge + 4 categories), and Error state ("Offline" badge). Vitest component tests in `client/tests/lab-01/App.test.tsx` passed 3/3 tests.

 [SCREENSHOT PLACEHOLDER: TokTickIT React App UI (Online Badge & Categories List)]

Figure 4: React UI rendering 'Online' status badge with the 4 seeded categories in Bootstrap list format

 [SCREENSHOT PLACEHOLDER: GitHub Project Board 'TokTickIT Individual Sprints']

Figure 5: GitHub Project Board showing 6 Kanban columns (Backlog through Done) with cards placed in PR Review and Done

Test Plan & Execution Evidence Table

#	Tool	Test File	Test Description	Result
1	Supertest	server/tests/lab-01/health.test.ts	GET /api/health returns 200 OK status	PASS
2	Vitest	client/tests/lab-01/App.test.tsx	Renders TokTickIT heading	PASS
3	Vitest	client/tests/lab-01/App.test.tsx	Shows Online badge & category list	PASS
4	Vitest	client/tests/lab-01/App.test.tsx	Shows Offline badge on network error	PASS

AI Use & Prompt Engineering Log (Claude Opus 4.6 & Gemini 3.6 Flash)

Established custom project governance rules in `_AI_EXECUTION_RULES.md` (such as the Ask-First protocol, mandatory peer review approvals before merge, markdown edit transparency, and dynamic mode toggles) alongside `docs/session_state.md`. As rules evolved across sessions, these living documents ensured that if I stop using Antigravity and switch to another AI tool altogether, any new AI assistant can immediately comprehend the full codebase history, git state, and execution standards without friction.